

The Role of Training Targets in Forgetting during Fine-Tuning of Time Series Foundation Models

Solgyu Lee¹, Jongsoo Choi²

¹*School of AI Software, Soongsil University, Seoul, Korea
leesg0107@gmail.com*

²*School of AI Convergence, Soongsil University, Seoul, Korea
jongsoo.choi@ssu.ac.kr*

Abstract

Foundation models are commonly fine-tuned to improve performance on specific downstream tasks. However, task-specific adaptation can degrade performance on non-target tasks, a phenomenon known as forgetting that undermines the broad generalization capabilities of foundation models. Prior studies on large language models have reported less forgetting under reinforcement fine-tuning (RFT) than under supervised fine-tuning (SFT), but the underlying mechanism remains unresolved. Isolating this mechanism is difficult in language-model reasoning tasks because multiple valid solution trajectories may yield the same final answer. By contrast, each time-series context window is paired with a single observed future trajectory, providing a more controlled setting for comparing the two paradigms. We therefore compare SFT with RFT instantiated using Group Relative Policy Optimization in two time series foundation models, Chronos-T5 and Moirai, using a systematically controlled family of loss configurations. At matched levels of target-task improvement, RFT exhibited significantly less forgetting than SFT in three of ten evaluable backbone-target cells, whereas SFT exhibited less forgetting in one cell; the remaining six comparisons were statistically inconclusive. We then decomposed the RFT objective by removing individual components from RFT and adding corresponding components to SFT. Among eight cells in which SFT produced at least 2% forgetting, seven showed a consistent separation based on whether the positive optimization target was model-generated or externally observed, rather than on the presence of any individual loss component. Furthermore, freezing model-generated targets after an initial adaptation phase showed no detectable reduction in retention relative to resampling them at every training step. These results indicate that the choice of optimization target plays a central role in forgetting during TSFM fine-tuning.

Keywords: Time Series Foundation Models, Time Series Forecasting, Forgetting, Reinforcement Fine-Tuning, Supervised Fine-Tuning

1. Introduction

In artificial intelligence, large-scale pretrained foundation models are valuable because a single model can support a broad range of downstream tasks. Fine-tuning is commonly used to improve performance on a task of interest. However, fine-tuning can degrade performance on tasks outside the fine-tuning target [1], a

phenomenon known as forgetting [2]. Forgetting undermines the broad generalization capabilities of foundation models, so it must be addressed as long as fine-tuning is used. Existing remedies either replay data from previously learned tasks during training [3] or constrain parameter updates or output-space deviations. If the source of forgetting were known, however, fine-tuning objectives could be designed to mitigate it directly.

Two widely used fine-tuning paradigms for adapting foundation models are supervised fine-tuning (SFT) and reinforcement fine-tuning (RFT). Prior studies have reported that RFT induces less forgetting than SFT in large language models (LLMs) [4,5]. The mechanism underlying this difference remains unresolved. The proposed explanations are difficult to distinguish empirically because the two methods cannot be placed on equal footing in LLMs. SFT trains toward a single answer recorded in the dataset. In RFT, the model generates candidate outputs, scores each by how close it is to the answer, and trains toward the high-scoring ones. The difficulty is that in science and mathematics problems the final answer is unique while the correct solutions leading to it are many. For RFT this multiplicity makes it difficult to determine which of the correct solutions the model converged toward. Because the two methods draw their targets from different sources from the outset, the two explanations cannot be separated in this setting.

Time series forecasting provides a more controlled setting. Each context window is paired with a single observed future trajectory, substantially reducing the ambiguity encountered in LLM reasoning tasks. All configurations use the same input windows and observed future values, while sampling, target selection, and reward-based weighting are varied systematically through the loss construction, which is therefore the only intentional difference among them. Forgetting is measured as the degradation in forecast accuracy on six evaluation domains that are not the fine-tuning target. On domains related to the target, transfer from the fine-tuning data can mask or mimic degradation, so forgetting is assessed on non-overlapping evaluation datasets against the same pretrained checkpoint. Observing that one configuration forgets less is not enough on its own, because it may have deviated less from the pretrained model. Time series foundation models (TSFMs) output a probability distribution over future values, so output-space displacement from the pretrained model can be measured directly by Kullback-Leibler (KL) divergence. Forgetting is therefore compared under two conditions: matched target-task gain and matched output-space displacement. In this study, RFT is instantiated using Group Relative Policy Optimization (GRPO), a policy-optimization method that estimates relative advantages from groups of sampled outputs without using a separate critic [6]. The two TSFM backbones are Chronos-T5 [7] and Moirai [8].

This comparison has not been carried out in TSFMs. A study that applies RFT to them evaluates only the fine-tuning target and nearby domains [9], and one that measures forgetting compares continual-learning strategies [10]. To our knowledge, no prior study has treated the loss construction as the primary variable in a controlled comparison of forgetting in TSFMs.

The contributions of this paper are twofold.

- 1) We conduct a controlled comparison of SFT and RFT in TSFMs, leveraging the single observed target associated with each forecasting context. The loss construction is treated as the primary experimental variable, and forgetting is evaluated on datasets that do not overlap with the fine-tuning target.
- 2) We decompose the RFT objective to examine which loss components are associated with reduced forgetting. In seven of the eight qualifying backbone-target cells, the configurations separate according to whether the positive optimization target is model-generated or externally observed. We further show that the observed retention effect does not require resampling the target at every training step.

The paper is organized as follows. Section 2 reviews prior work. Section 3 defines the loss functions of the two methods together with forgetting and displacement. Section 4 describes the experimental setup and the loss configurations. Section 5 analyzes the results and discusses implications and limitations. Finally, Section 6 concludes the paper and outlines the next steps for further research.

2. Related Work

Explanations proposed for forgetting. The explanations offered for why RFT forgets less than SFT attribute reduced forgetting to different mechanisms within RFT. One holds that the amount of forgetting is determined by the KL divergence from the original model [4]. On this view, if two models have moved the same distance, the method used to train them does not matter. That study varies the method and observes the distance, and in one analysis regularizes each method toward the pretrained model with a KL penalty. It does not place two losses at a common measured displacement and compare forgetting at that displacement. The other holds that how the reward is handled is secondary, and that the essential fact is that the training data is the model’s own output [5]. That study also reports that data regenerated at the start of each epoch, rather than at every step, can suffice to mitigate forgetting. The two explanations have not been reconciled, and a regime in which the predicted monotonic relationship does not hold [5]. The two explanations imply different algorithmic strategies. When the proposed explanatory variables covary, however, the two explanations cannot be distinguished empirically. Determining which part produces the retention requires decomposing RFT component by component rather than comparing it as a whole.

Studies that decompose the loss. The RFT loss is built from several components such as sampling, reward comparison, reward-based weighting, and a term that lowers the probability of low-reward samples. The work that proposed GRPO already organized SFT and several reinforcement learning methods into a single gradient form [6], and the configurations used here are instances of that framework. A method that selects only the highest reward sample and places it in the standard loss has also been proposed [11]. It was proposed to simplify alignment, and performance on tasks outside the aligned objective was not measured. Here the same form serves as the minimal configuration that pulls toward the model’s own output. A study that isolates the term lowering the probability of incorrect samples argues that this term refines rather than replaces the model’s existing knowledge, a claim derived analytically and evaluated through output diversity within the fine-tuned task [12]. That decomposition is defined for binary rewards. Here the corresponding term is defined by the sign of a group-standardized advantage under a continuous reward, and the configurations move it across losses and measure retention outside the task. Another study measures algorithm-level substitutions together with retention outside the target task, yet the substitution is made at the algorithm level, the setting still admits many correct solutions, and attribution to the individual terms of the loss remains open [5]. Fixing displacement as a budget has been used to measure the amount of optimization within a single method, and the study that popularized this practice explicitly warns against using it to compare different methods, because they spend displacement differently [13]. To our knowledge, no prior study has combined component decomposition, displacement matching, and measurement of retention outside the target task.

Fine-tuning and forgetting in time series foundation models. One study applies RFT to these models but restricts evaluation to the fine-tuning target or to nearby domains [9]. Another addresses forgetting and does measure degradation, but in its ablation the compared variables are the construction of the replay samples and an alignment constraint, and degradation is reported as an average over all old tasks [10]. The evaluation domains in this paper are not domains the model has never seen. They are domains that do not overlap with the fine-tuning target, and degradation is defined as a relative change against the same pretrained checkpoint.

What this study does not address. The measurements here remain at the level of the output distribution. Matching output-space displacement does not constrain the location of parameter updates, and the influence of the pretraining stage on post-fine-tuning forgetting is not tested either. Differences between backbones with distinct pretraining recipes should therefore not be interpreted as effects of the loss function.

3. Preliminaries

3.1 Notation

Fine-tuning takes place on the task of receiving a context window x and predicting the future that follows it. The observed future values are denoted by y , the predictive distribution of the model being fine-tuned is written π_θ , and the pretrained checkpoint before fine-tuning begins is written π_0 . Forgetting is defined as

degradation relative to π_0 , so π_0 is the reference point for every comparison.

The two backbones represent the predictive distribution differently. Chronos-T5 quantizes the future values y into a token sequence z over K bins and predicts the tokens one at a time over the H positions of the horizon. The predictive probability of y is represented by the probability assigned to its tokenized sequence z . The predictive distribution of the fine-tuned model is defined as

$$\pi_\theta(y | x) = \prod_{h=1}^H p_\theta(z_h | x, z_{<h}), p_\theta(\cdot | x, z_{<h}) \in \Delta^{K-1} \quad (1)$$

where z_h is the token at position h and $z_{<h}$ are the tokens before it. Each factor is a distribution over the K bins, and Δ^{K-1} denotes the set of such distributions. Throughout the paper, p_θ denotes a distribution at a single position and π_θ the distribution over the whole horizon, with the subscript naming the model. Moirai models the value at each time step directly as a mixture distribution.

$$\pi_\theta(y | x) = \prod_{h=1}^H p_\theta(y_h | x), p_\theta(y_h | x) = \sum_{c=1}^C w_c(x) f_c(y_h; \phi_c(x)) \quad (2)$$

where y_h is the value at position h , w_c are the mixture weights, f_c are the component densities with parameters ϕ_c , and C is the number of mixture components. The weights and parameters depend on the position as well, and the position index is suppressed. Thus, Chronos-T5 represents the predictive distribution over a discrete grid, whereas Moirai uses a mixture of continuous densities.

3.2 Displacement

Model displacement from the pretrained checkpoint is quantified using the KL divergence from π_0 to π_θ , with the pretrained model as the first argument. This quantity is used exclusively for evaluation and does not appear in the training objective. For Equation (1) each position is a distribution over a finite set of categories, so the KL divergence at a single position has a closed form as

$$KL_h = \sum_{k=1}^K p_0(k | x, z_{<h}) \log \frac{p_0(k | x, z_{<h})}{p_\theta(k | x, z_{<h})} \quad (3)$$

where $z_{<h}$ is fixed to the tokens of the realized future values. Both models receive the same context and the same $z_{<h}$, so what is measured is the change in the predictive distribution itself rather than what each model would have generated. $p_0(\cdot)$ is the per-position distribution of the pretrained checkpoint.

For Equation (1), the displacement $D(\theta)$ is KL_h averaged over positions and contexts. For Equation (2) the KL divergence between the mixture distributions has no closed-form expression, so the divergence is estimated with samples $v^{(1)}, \dots, v^{(B)}$ drawn from π_0 as follows

$$\widehat{D}(\theta; x) = \frac{1}{B} \sum_{b=1}^B [\ell_0(v^{(b)} | x) - \ell_\theta(v^{(b)} | x)], v^{(b)} \sim \pi_0(\cdot | x) \quad (4)$$

where $\ell_\theta(u | x)$ denotes the log-likelihood of a series u under π_θ , averaged over the evaluated positions, and $\ell_0(u | x)$ denotes the same under π_0 . The average of these differences over the samples is an unbiased estimate of the divergence at the given context. Both log-likelihoods are evaluated on the same samples, thereby reducing Monte Carlo estimation variance. For Equation (2), $D(\theta)$ is $\widehat{D}(\theta; x)$ averaged over contexts.

3.3 Reward and Advantage

The reward is the negative squared error between the prediction and the realized values, averaged over the horizon. Because series differ in magnitude, the error is divided by a scale factor. The scale is the context scale of the tokenizer for Chronos-T5 and the mean absolute value of the realized future values for Moirai. The scale is constant within a window. It cancels in the group-standardized advantage up to the small constant in the divisor, leaves the highest reward sample unchanged, and no update used here depends on its choice.

For a given context, G samples $\hat{y}_1, \dots, \hat{y}_G$ are drawn from the current policy at every step. The reward of each sample is centered by the group mean and divided by the group standard deviation. A small constant added to the divisor keeps the division defined. The result is the advantage $\hat{A}_i$. A positive $\hat{A}_i$ means the sample is better than the average of the group drawn from the same context, and a negative one means it is worse.

The observed future is the only prediction with zero error and therefore constitutes the unique global maximizer of the reward. In Chronos-T5 the output lies on a quantization grid, so the unique maximizer is the quantized representation of the realized values. In an LLM the many solution paths reaching one answer all receive the maximum reward, and this is what makes the two methods incomparable there. In this setting, by contrast, no two distinct outputs attain the maximum reward, and the specific ambiguity associated with multiple maximally rewarded solution trajectories is absent.

3.4 Decomposing the Loss

The SFT and RFT objectives are defined as follows:

$$L_{SFT} = -\ell_{\theta}(\mathcal{Y} | x), \quad (5)$$

$$L_{RFT} = -\frac{1}{G} \sum_{i=1}^G \hat{A}_i \ell_{\theta}(\hat{y}_i | x). \quad (6)$$

The SFT objective in Equation (5) raises the probability of the realized values recorded in the dataset, and the model’s own output never enters the loss. The RFT objective in Equation (6) raises or lowers the probability of each sample the model has drawn, according to the sign of the advantage and in proportion to its magnitude. A single gradient step is taken on each rollout batch, so the importance ratio is one and lies inside the clipping range. The surrogate objective of GRPO [6] therefore gives the same gradient as Equation (6), and it is used from here on. No KL penalty against a reference policy is applied. Writing $\hat{A}_i^+ = \max(\hat{A}_i, 0)$ and $\hat{A}_i^- = \min(\hat{A}_i, 0)$ rewrites Equation (6) as the sum of two terms defined as

$$L_{RFT} = -\frac{1}{G} \sum_{i=1}^G \hat{A}_i^+ \ell_{\theta}(\hat{y}_i | x) + \frac{1}{G} \sum_{i=1}^G |\hat{A}_i^-| \ell_{\theta}(\hat{y}_i | x). \quad (7)$$

The first term raises the probability of samples better than average and is called the pull term. The second lowers the probability of samples worse than average and is called the push term.

3.5 Forgetting

Forgetting is measured with the mean absolute scaled error (MASE). MASE divides the mean absolute error of the point forecast by that of the seasonal naive forecast computed on the context window. The point forecast is the median of the predictive distribution. Forgetting on evaluation domain d is defined as the relative increase in domain-level mean MASE:

$$F_d(\theta) = \frac{\text{MASE}_d(\pi_{\theta}) - \text{MASE}_d(\pi_0)}{\text{MASE}_d(\pi_0)} \times 100 \quad (8)$$

where $\text{MASE}_d(\pi)$ denotes the average over all evaluation windows of domain d . Higher values of F indicate greater forgetting, whereas negative values indicate improved performance on the evaluation domains relative to the pretrained checkpoint.

4. Experimental Setup

4.1 Backbones and Data

The two backbones were chosen so that their representations of the predictive distribution differ. We use

Chronos-T5 with 46M parameters and Moirai with 14M parameters. For both backbones, all parameters are updated, and no adapters are used. Each backbone is fine-tuned on all six targets, giving twelve combinations in total. Each combination of a backbone and a target is called a cell.

We consider six fine-tuning targets: electricity transformer temperature (ETTh1), electricity demand (AED), tourism demand (tourism_q), river flow (saugenday), and the weekly (m4_weekly) and yearly (m4_yearly) series of the M4 competition. They were chosen so that the sampling intervals range from hourly to yearly, and the forecast horizons range from 6 to 48 steps. The purpose is to show that the results are not confined to one or two domains.

Forgetting is measured on six domains that are not the fine-tuning target, hereafter referred to as the evaluation domains. They are electricity consumption (electricity), road occupancy (traffic), solar power output (solar), pedestrian counts (pedestrian), weather observations (weather), and hospital admissions (hospital). None of their datasets overlaps with a fine-tuning target. The evaluation windows are cached in advance so that every training run uses the same ones. Each evaluation domain contributes 32 series with three windows each, except the hospital dataset, for which two windows per series are used. Predictive quantiles are estimated from 20 samples for Chronos-T5 and 32 for Moirai.

4.2 Loss Configurations

Six configurations are built using the decomposition of Equation (7). All configurations use the same training data. Sampling-based configurations use the same reward definition, whereas target selection, advantage weighting, and the push term vary across the training objectives. Each sampling-based configuration draws $G = 8$ candidate forecasts per context. Table 1 gives the loss of each configuration alongside what it targets and which components it retains.

Table 1. Loss configurations

Configuration	Loss	Target of the loss	Sampling	Advantage weighting	Push term
SFT	$-\ell_{\theta}(\mathcal{Y} x)$	Dataset value	None	None	None
RFT	$-\frac{1}{G} \sum_{i=1}^G \hat{A}_i \ell_{\theta}(\hat{\mathcal{Y}}_i x)$	G model-generated samples	Every step	Yes	Yes
RFT-pull	$-\frac{1}{G} \sum_{i=1}^G \hat{A}_i^+ \ell_{\theta}(\hat{\mathcal{Y}}_i x)$	G model-generated samples	Every step	Positive only	None
SFT-push	$-\ell_{\theta}(\mathcal{Y} x) + \frac{1}{G} \sum_{i=1}^G \hat{A}_i^- \ell_{\theta}(\hat{\mathcal{Y}}_i x)$	Dataset value	Every step	None	Yes
RAFT	$-\ell_{\theta}(\hat{\mathcal{Y}}^* x)$	Best-reward model-generated sample	Every step	None	None
RAFT-once	$-\ell_{\theta}(\hat{\mathcal{Y}}^{\text{fix}} x)$	Best-reward model-generated sample	Once during training	None	None

In Table 1, $\hat{\mathcal{Y}}^*$ is the highest reward sample among the G drawn at that step. $\hat{\mathcal{Y}}^{\text{fix}}$ is the highest reward sample among the G drawn once from the policy at step T_0 , one-sixth of the way through training, and is

held fixed thereafter. Before T_0 the configuration trains as reward ranked fine-tuning (RAFT) [11]. The only difference between RAFT and RAFT-once is whether the target is redrawn at every step or drawn once. SFT and RAFT have the same loss form and differ only in what the loss raises the probability of. SFT-push has one property that the loss expression alone does not reveal. Computing $\hat{A}_i$ requires drawing samples, so the model’s own output enters the loss in this configuration as well. It enters only in the direction that lowers probability.

4.3 Training and Checkpointing

All configurations share the same optimization setup. All models are optimized with AdamW. Moirai is trained for 3,600 steps, whereas Chronos-T5 is trained for 3,000 steps. Each backbone-target cell is evaluated with three random seeds. Only the learning rate is swept separately for each configuration. Using a single learning rate would confound the comparison because the effective update magnitude differs across configurations.

Checkpoints are not saved at fixed step intervals. They are saved the moment a predetermined displacement level is first crossed. Saving at step intervals would leave no points at which two configurations have reached the same displacement, and comparison at matched displacement would not be possible. Displacement is evaluated on a fixed set of 128 contexts drawn from the validation split of the fine-tuning target for every cell. For Moirai, the estimate of Equation (4) uses $B = 64$ samples. Performance on the evaluation domains and on the target domain is measured only at the saved checkpoints.

4.4 Measurement Protocol

Comparing at equal task gain. One comparison controls the gain on the target domain rather than the displacement. The gain on the target τ , written $\text{Gain}_\tau(\theta)$, is Equation (8) computed on τ instead of an evaluation domain, with the two terms in the numerator reversed so that a larger value means more improvement. It is measured on held-out validation windows of the target. The reference $\text{MASE}_\tau(\pi_0)$ is measured zero-shot on the same validation windows. A negative gain means the checkpoint performs worse on the target than the pretrained model. The frontier of a configuration a is defined as follows.

$$\Phi_a(g) = \min\{F(\theta) : \theta \in \Theta_a, \text{Gain}_\tau(\theta) \geq g\} \quad (9)$$

where Θ_a is the set of saved checkpoints of configuration a , and $\Phi_a(g)$ is the least forgetting attainable while achieving a gain of at least g . The inequality defines $\Phi_a(g)$ as a minimum over every checkpoint whose gain reaches g , so the frontier is monotone in g and does not depend on the checkpoint nearest to g . A minimum over a larger pool can still be lower, and the RFT pools are larger. Subsampling the larger pool to the size of the smaller and averaging over resamples changes no comparison, indicating that the ordering is not attributable to the difference in pool size. The comparison is defined only within the gain range that both configurations reached. The lower bound is the larger of the two minima and the upper bound is the smaller of the two maxima. The comparisons outside this common range would require extrapolation and are therefore excluded. The reported number divides that range into five equally spaced points, evaluates $\Phi_{\text{SFT}}(g) - \Phi_{\text{RFT}}(g)$ at each, and averages them. Frontiers for the reported statistics are built separately for each seed and pool the checkpoints of all learning rates within that seed.

Comparing at equal displacement. The component comparisons fix displacement. This analysis tests the hypothesis that forgetting is fully explained by output-space displacement under the condition most favorable to that hypothesis. A residual difference at approximately matched displacement indicates that displacement alone is insufficient to explain the observed forgetting. Displacement is not used to compare the amount of training across methods, which the study that popularized displacement budgets warns against [13].

The overlapping range is taken the same way. Displacement spans orders of magnitude and the saved levels

are themselves geometrically spaced. The range is divided into five logarithmically spaced points, and the nearest-checkpoint selection is also made in log coordinates. At each point, F is taken from the closest saved checkpoint of each configuration and the differences are averaged. The choice of protocol is a degree of freedom that can change results. The logarithmic readout is the primary one and the reported statistics use it. Using arithmetic rather than logarithmic coordinates did not reverse the direction of any comparison. Where a seed's overlapping range is narrow the selected checkpoint shifts and the variance across seeds grows. In m4_yearly and saugeenday with Moirai and in ETTh1 with Chronos-T5 some intervals exclude zero under one readout and contain it under the other.

Cells that can be evaluated. Where SFT reaches no positive gain on the target, as in tourism_q and m4_weekly with Chronos-T5, there is no adaptation against which forgetting can be read, and the cell is excluded. Why adaptation fails in a given cell is not the question this study addresses. The component comparisons are restricted to cells in which the largest SFT forgetting across all checkpoints saved for the twelve-cell comparison, over all learning rates and seeds, reaches at least two percent. Eight cells qualify: all six targets with Moirai and ETTh1 and AED with Chronos-T5. The qualifying cells reach at least 2.09 percent and the excluded cells at most 1.77. The threshold was set after inspecting the runs, and this is recorded as a limitation.

Statistics. Comparisons between configurations are reported using 95 percent confidence intervals based on Student's t distribution over three independent training runs. An effect is considered statistically distinguishable from zero when the corresponding interval excludes zero. Resampling windows within a single run treats correlated windows as independent and understates the variance. All reported confidence intervals are constructed at the run level. The number of positions at which displacement is evaluated, the presence of autoregressive conditioning, and the scale used to normalize the reward all differ by backbone. The absolute values of displacement and reward are therefore not compared across backbones. Conclusions are interpreted within each backbone only, and contrasts between backbones are presented as observations without assigning a cause.

5. Results and Discussion

5.1 Comparison of RFT and SFT at Matched Target-Task Gain

The two methods were compared at points of equal task gain. Figure 1 shows three representative targets for each backbone. Each panel plots the gain the two methods reached and the forgetting at that gain. In the evaluable cells the axes are cropped to the gain band that both methods reached. Each point is one checkpoint, and its height is F . The curves pool the checkpoints of all seeds.

The shaded band in each panel in Figure 1 is the gain range that both methods reached. In the tourism_q panel of Chronos-T5, SFT achieves no positive gain; therefore, no comparison at matched positive gain is possible, and the cell is excluded. The panel is kept as an example of the failure mode. In m4_yearly with Chronos-T5 only one seed has an overlapping range, so no interval is formed, and that cell is not counted either. RFT forgets less than SFT in three cells, by 6.84, 1.15, and 0.40 percentage points on ETTh1 with Moirai, AED with Moirai, and ETTh1 with Chronos-T5, respectively. SFT forgets less in one cell, saugeenday with Chronos-T5, by 0.25 percentage points. These four intervals exclude zero. The remaining six evaluable cells are inconclusive.

5.2 The Training Target, Rather Than an Individual Loss Component, Separates the Outcomes

RFT differs from SFT in four ways at once. It weights samples by reward, lowers the probability of low-reward samples, draws samples at every step, and raises the probability of its own samples. Identifying what produces the retention therefore requires removing them and observing what survives. The configurations built from the decomposition of Equation (7) were compared at equal displacement. Figure 2 shows the result for the eight qualifying cells.

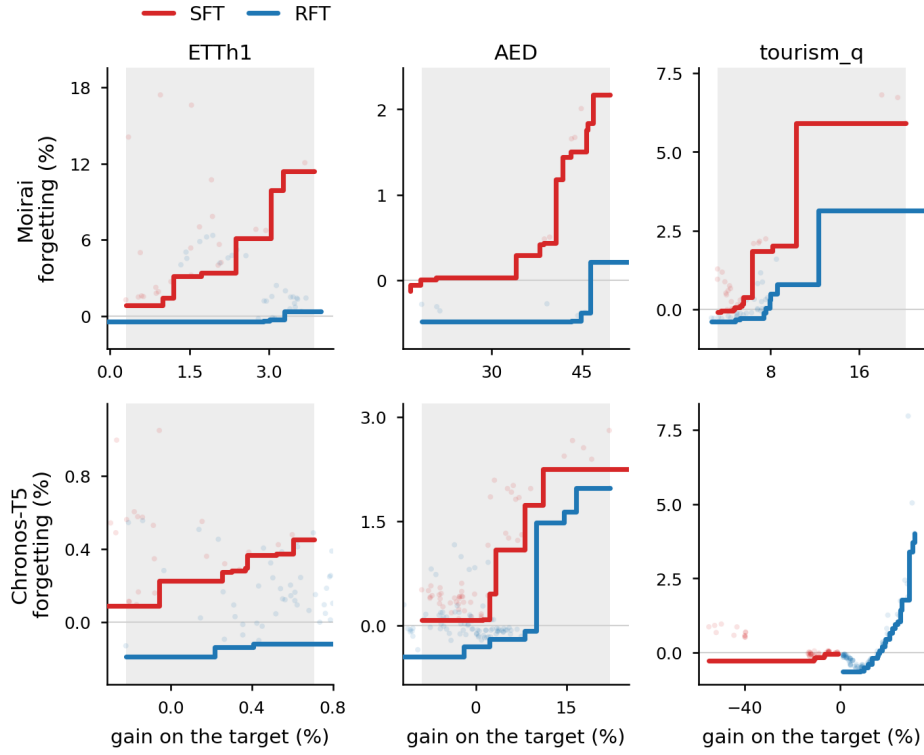


Figure 1. Forgetting at equal task gain

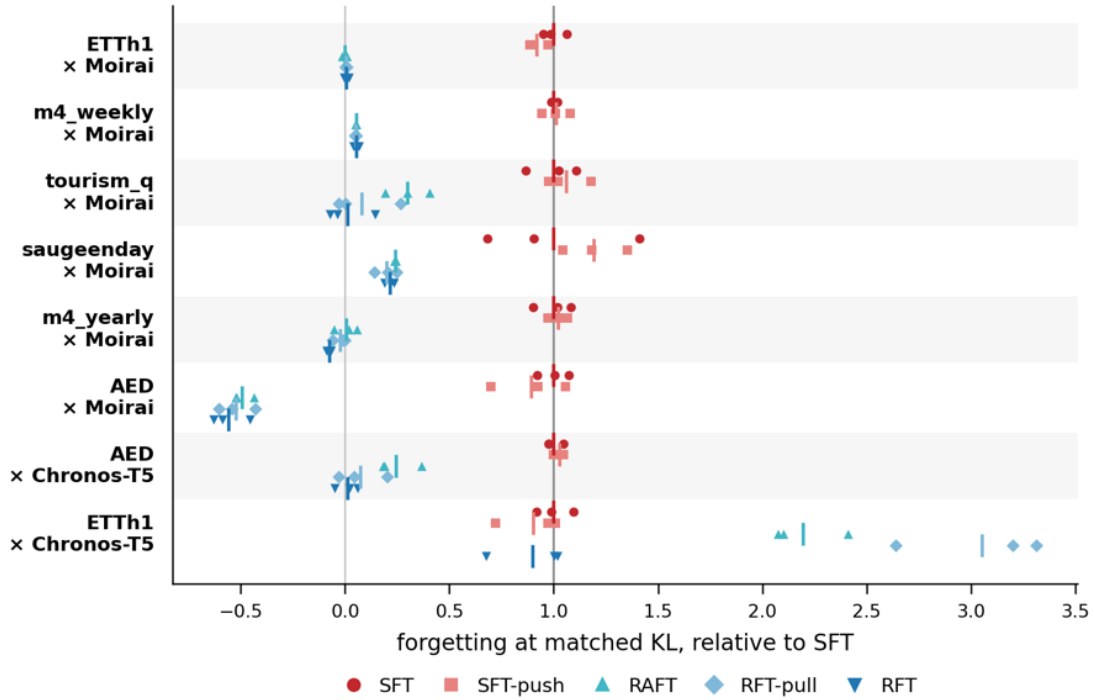


Figure 2. Forgetting at matched displacement, relative to SFT

RAFT-once changes only when the target is drawn and is examined separately. In Figure 2, the horizontal axis reports the ratio of forgetting under each configuration to forgetting under SFT at the same displacement. A ratio of 1 indicates forgetting equal to that of SFT, 0 indicates no forgetting, a negative ratio indicates improvement on the evaluation domains, and a ratio greater than 1 indicates more forgetting than under SFT. The numbers quoted in the text are differences in percentage points, not these ratios.

Effect of removing the push term. RFT-pull removes low-reward samples from the loss entirely, so only the positive side of the weighting remains. The retention effect persists after removal of the push term. It forgets less than SFT on all six Moirai targets, by 9.88 and 8.41 percentage points on ETTh1 and m4_weekly, respectively, and every interval excludes zero.

Effect of removing advantage weighting. RAFT uses only the single highest reward sample, with neither weighting nor pushing. It is the simplest configuration of all, yet a comparable retention effect is observed. The values with Moirai are 10.37 and 9.20 percentage points on ETTh1 and m4_weekly, respectively, comparable in magnitude to that of RFT itself at 10.90 and 8.50.

Effect of sampling when the positive target is not model-generated. SFT-push trains toward the realized values of the dataset while retaining the push term unchanged. This configuration also draws samples from the model, because computing the negative part of the advantage requires them. The model's own output is therefore present inside the loss. Even so, this configuration showed no detectable retention advantage over SFT: none of the six Moirai comparisons was statistically distinguishable from zero, and the gaps ranged from -0.49 and $+0.13$ percentage points.

Role of the optimization target. RFT, RFT-pull, and RAFT share the property that their objectives increase the probability of the model-generated outputs. SFT-push is the one in which a dataset value occupies that position. Having the model's own samples appear in the loss is not enough. Across the tested configurations, a detectable retention advantage was observed only when model-generated samples served as positive optimization targets. The strongest retention appears on AED with Moirai, where RFT, RFT-pull, and RAFT lie near -0.5 on the axis of Figure 2. At matched displacement these configurations do not merely forget less than SFT. The evaluation domains improve by about 0.33 percent.

Backbone-specific behavior in Chronos-T5. On AED the ordering matches the Moirai cells. On ETTh1 it reverses. At matched displacement RAFT forgets more than SFT by 1.18 percentage points with interval $[0.54, 1.82]$, and RFT-pull by 2.09 with interval $[1.43, 2.74]$, while RFT and SFT-push show no statistically significant difference from SFT. The reversal holds when the checkpoint set is extended to higher learning rates and higher displacement levels, and under the arithmetic readout, but not when the controlled variable changes. When the gain is fixed instead of the displacement, the RAFT gap becomes -0.11 with interval $[-0.22, -0.01]$. The reversal is therefore not an artifact of the checkpoint set or of the readout, and it is a property of the fixed displacement readout. What distinguishes this cell in the recorded runs is that it is the cell in which SFT achieves the smallest gain, 0.71 percent, compared with at least 3.86 percent in the other seven cells. We do not attribute this behavior to a specific mechanism.

5.3 Redrawing the Target at Every Step Is Not Required

The remaining question is whether the target must be redrawn at every step. RAFT-once draws the samples once during training, freezes them, and carries out the rest of training with those alone. Figure 3(a) shows the full trajectory of one cell, ETTh1 with Moirai, over the displacement axis, and the two curves overlap across the whole range. Figure 3(b) summarizes all eight cells at matched displacement on the same relative axis as Figure 2. The six Moirai gaps lie between -0.09 and $+0.02$ percentage points. The two Chronos-T5 cells give $+0.06$ with interval $[-0.29, +0.42]$ and $+0.71$ with interval $[-0.57, +1.99]$. Seven of the eight intervals contain zero. The remaining one, m4_yearly with Moirai, spans $[-0.03, -0.01]$, a gap of -0.02 percentage points against a total SFT forgetting of 1.35 percent in that cell. Freezing model-generated targets after the initial adaptation phase produced retention outcomes comparable to those obtained by resampling the targets at every

step, within the statistical resolution of the present experiments. This sharpens a prior observation that approximately on-policy data regenerated at the start of each epoch can suffice [5].

We also evaluated a configuration in which targets were sampled from the pretrained policy and then frozen. Because these targets already had high likelihood under the pretrained model, matching their displacement to that of the other configurations required substantially higher learning rates, thereby confounding target-generation time with optimization scale. This configuration was therefore excluded from the primary comparison.

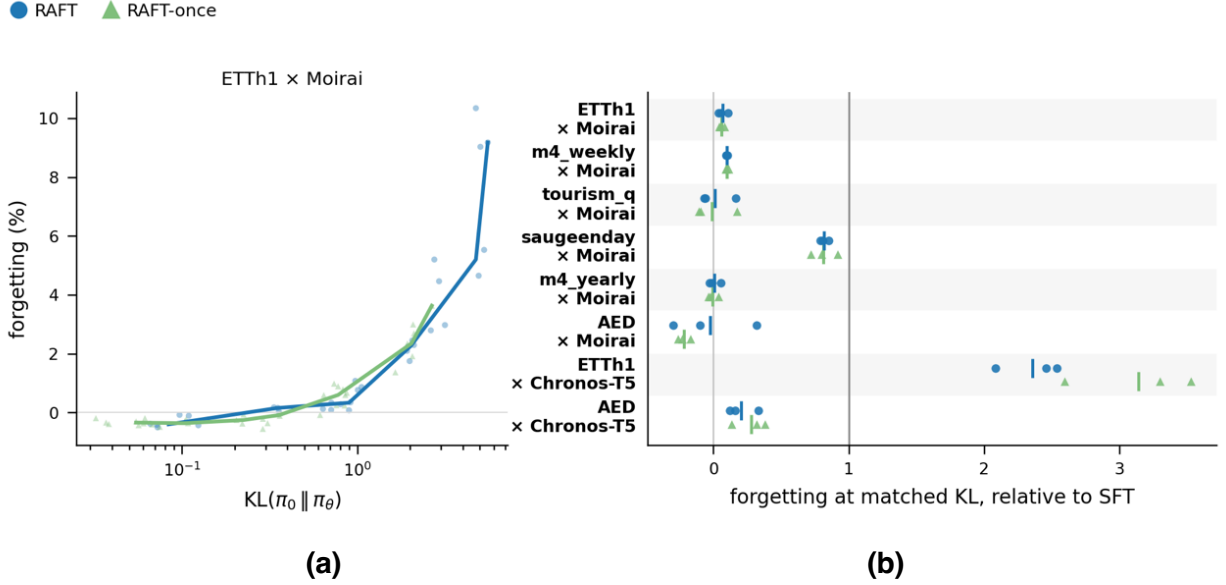


Figure 3. Comparison of repeatedly resampled and once-sampled frozen targets

5.4 Discussion and Limitations

Within the present experimental design, the results indicate that forgetting is not fully explained by output-space displacement alone and are more consistent with a target-based account. The explanation that the amount of forgetting is determined by distance from the original model [4] does not fit the observation that gaps remain at points where distance has been matched. Those gaps exceed the gaps between configurations that share a target in every one of the seven cells, by roughly an order of magnitude in the two whose values are quoted above. The explanation that the essential fact is that the training data is the model’s own output [5] points in the same direction as these results. This study supports it in a setting where multiple correct solutions have been removed, with the one exception noted above.

Five limitations remain. First, this study investigates, in time series forecasting, a question that originated in LLM adaptation. Each forecasting context is associated with a **unique** observed target; whether the same target-dependent separation holds in settings with multiple valid solution trajectories remains open. Second, the component comparisons are restricted to cells in which SFT causes at least two percent forgetting, and the conclusions drawn from them do not extend to cells where it does not. Third, the run-level statistics are based on three independent training runs, so the power is low. An interval containing zero does not mean there is no difference. Rather, it indicates that the available runs provide insufficient evidence to resolve the difference. Fourth, the two percent threshold used to select cells for the component comparison was determined after inspecting the experimental results. Consequently, this post hoc selection rule limits the generalizability of the component-level findings and may introduce selection bias. Fifth, the origin of the target is confounded with

its statistical character. The realized values carry observation noise, whereas a sample from the policy is drawn from the model's own predictive distribution. An external target with the noise removed, or a sample drawn from a different model, would separate the two, and neither control was evaluated. Accordingly, the evidence supports an association between target origin and retention rather than establishing target origin as the sole causal factor.

6. Conclusion

Fine-tuning can degrade performance on tasks outside the adaptation target. Prior studies have repeatedly reported less forgetting under RFT than under SFT in LLMs, but the underlying mechanism remains unresolved because the two methods are difficult to compare under fully controlled conditions. This study examines this comparison in time series forecasting, where each context is associated with a single observed future trajectory.

Decomposing the RFT loss into its components shows that what divides forgetting is not which components are present, but what the loss raises the probability of. One Chronos-T5 cell deviates from this overall pattern. Removing the reward weighting leaves the retention intact and so does removing the term that pushes down low-reward samples. Moving that term into SFT and making it draw samples does not produce retention. Keeping the two losses in the same form, as in SFT and RAFT, and changing only the value the loss pulls toward produces the retention. The retention also survives freezing the target. Models trained with targets frozen after the initial adaptation phase retained prior capabilities comparably to models trained with targets resampled throughout training. These findings suggest that reduced forgetting may be achieved without retaining every component of the RFT objective, provided that model-generated outputs serve as the positive optimization targets.

The next step is to test this conclusion in the setting where the question originated. Time series forecasting is not the only setting in which a single model holds many abilities. The same degradation appears in LLMs [2] and in vision-language-action models, where gradients from a newly initialized action expert degrade the language-following capability of the pretrained backbone [14], and those settings differ from this one in that many paths lead to the correct answer. Whether this target-dependent pattern generalizes to settings with multiple valid solution trajectories, including LLMs and vision-language-action models, remains an important direction for future research.

Acknowledgement

This research was supported by the Ministry of Science and ICT(MSIT), Korea and the Institute of Information & Communications Technology Planning & Evaluation(IITP) under AI University(2026-0-00057, 2026).

References

- [1] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35, pp. 27730-27744, December 2022.
- [2] Y. Luo, Z. Yang, F. Meng, Y. Li, J. Zhou, and Y. Zhang, "An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-Tuning," *IEEE Transactions on Audio, Speech and Language Processing*, Vol. 33, pp. 3776-3786, September 2025.
- [3] T. Scialom, T. Chakrabarty, and S. Muresan, "Fine-Tuned Language Models Are Continual Learners," in *Proc.*

- Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6107-6122, December 2022.
- [4] I. Shenfeld, J. Pari, and P. Agrawal, "RL's Razor: Why Online Reinforcement Learning Forgets Less," in *Proc. International Conference on Learning Representations (ICLR)*, April 2026.
 - [5] H. Chen, N. Razin, K. Narasimhan, and D. Chen, "Retaining by Doing: The Role of On-Policy Data in Mitigating Forgetting," in *Proc. International Conference on Machine Learning (ICML)*, July 2026.
 - [6] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, February 2024.
 - [7] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. Pineda Arango, S. Kapoor, J. Zschiegner, D. C. Maddix, H. Wang, M. W. Mahoney, K. Torkkola, A. G. Wilson, M. Bohlke-Schneider, and Y. Wang, "Chronos: Learning the Language of Time Series," *Transactions on Machine Learning Research*, October 2024.
 - [8] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, "Unified Training of Universal Time Series Forecasting Transformers," in *Proc. International Conference on Machine Learning (ICML)*, pp. 53140-53164, July 2024.
 - [9] S. Li, Y. Chen, Z. Zhu, Y. Pan, Y. Guo, M. Huang, and H. Xiong, "TimeRFT: Stimulating Generalizable Time Series Forecasting for TSFMs via Reinforcement Finetuning," *arXiv preprint arXiv:2605.00015*, May 2026.
 - [10] T. Yin, J. Wang, C. Wang, H. Wang, J. Cai, M. Liu, Y. Ma, K. Gao, Y. Song, and W. Shen, "R-Tuning: Wavelet-Decomposed Replay and Semantic Alignment for Continual Adaptation of Pretrained Time-Series Models," in *Proc. AAAI Conference on Artificial Intelligence*, Vol. 40, No. 33, pp. 27870-27878, January 2026.
 - [11] H. Dong, W. Xiong, D. Goyal, Y. Zhang, W. Chow, R. Pan, S. Diao, J. Zhang, K. Shum, and T. Zhang, "RAFT: Reward rAnked FineTuning for Generative Foundation Model Alignment," *Transactions on Machine Learning Research*, November 2023.
 - [12] X. Zhu, M. Xia, Z. Wei, W.-L. Chen, D. Chen, and Y. Meng, "The Surprising Effectiveness of Negative Reinforcement in LLM Reasoning," in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 38, December 2025.
 - [13] L. Gao, J. Schulman, and J. Hilton, "Scaling Laws for Reward Model Overoptimization," in *Proc. International Conference on Machine Learning (ICML)*, Vol. 202, pp. 10835-10866, July 2023.
 - [14] D. Driess, J. T. Springenberg, B. Ichter, L. Yu, A. Li-Bell, K. Pertsch, A. Z. Ren, H. Walke, Q. Vuong, L. X. Shi, and S. Levine, "Knowledge Insulating Vision-Language-Action Models: Train Fast, Run Fast, Generalize Better," in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 38, December 2025.